

■ 4.1.4 Functional Programming

An interactive system like *Mathematica* gives you the possibility of building up your programs in stages, rather than having to write a whole program in one go. Some styles of programming make this kind of incremental development easier than others.

Procedural programs, for example, are not particularly easy to build in stages. At any given point in a procedural program, there will usually be many local variables defined, and various control structures in use. As a result, it is not particularly easy to just “slot in” another piece of code.

One approach that works well for building up programs in stages is *functional programming*. The basic idea is to have your program consist of many nested functions, each one acting on results from previous ones. Instead of defining local variables to store your intermediate results, you just pass the results immediately to other functions.

```

exprod[n_Integer] :=
  Block[ { t, i } ,
    t = 1 ;
    For [ i = 0, i < n, i++,
      t = ( x + i ) t
    ] ;
    t = Expand[t] ;
    Return[t] ;
  ]

```

A procedural program for expanding out a product of terms.

```

exprod[n_Integer] := Expand[ Product[x + i, {i, n}] ]

```

A functional program for expanding out a product of terms.

In simple cases, you can think of functional programs as just applying sequences of functions to a single “current expression”. All the “state” of your program is encapsulated in the current expression. If you want to add a new piece to your program, you simply have to insert another function to apply to the current expression. If you want to know the intermediate results of a functional program, all you need do is to print out the “current expression” at a particular stage in the execution of the program. Unlike procedural programs, functional programs have no “hidden state” associated with the values of local variables in use, or the control structures being executed.

In building up a functional program, it is important that the current expression

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

can have a nontrivial structure. In the classic example of APL, the “current expression” is a nested list of numbers. In the pure LISP language, the current expression is an arbitrary list structure.

If you follow the functional programming style, you should rarely need to use local variables or control structures. You can often avoid control structures by using iteration functions like `Sum`. You can take a function that you have built up, and then apply a function like `Sum` to iterate over different instances of it.

When you write complicated programs in the functional programming style, you will find that you need to nest functions many levels deep. It is important to format your code so that the nesting is clear. One way to do this is simply to indent blocks at successively higher nesting levels.

Pure Functions

When you build up a functional program, you will often need to use *Mathematica* functions like `Apply`, `Map` and `Array` which take *functions* as arguments, and apply them in various ways to the current expressions. There are various ways to specify the functions you give as arguments. If they are built-in functions, or are functions that you have explicitly defined, then you can refer to them simply by giving their names. In many cases, however, it is more convenient to be able to build up “pure functions” to which you do not have to give specific names.

You can create pure functions in *Mathematica* using `Function` (abbreviated `&`). An expression like `Function[x, x^2 + 3]` represents a pure function which takes its argument, squares it, and adds 3 to the result. This kind of *Mathematica* expression is directly analogous to the λ expressions that are used in LISP or in formal logic.

When you write large functional programs, they usually contain many pure functions. *Mathematica* has various short-hand notations for pure functions. You can, for example, write `Function[x, x^2 + 3]` as `Function[#^2 + 3]` or even `(#^2 + 3)&`.

One goal of functional programming is completely to eliminate named variables. When you use a function like `Sum` to implement iteration, you still have to give an explicit name to the iteration variable that you use. You can do iteration without introducing any variables at all by using `Nest` and `NestList`.

Here is a pure function for building up products of the form $x(x+1)(x+2)\dots$

```
In[1]:= prod = {1 + First[#], Apply[Times, #]} &
Out[1]= {1 + First[#1], Apply[Times, #1]} &
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

This is the result from one application of `prod`.

```
In[2]:= prod[{x, 1}]
Out[2]= {1 + x, x}
```

This is the result of applying `prod` again.

```
In[3]:= prod[%]
Out[3]= {2 + x, x (1 + x)}
```

Here is a functional program that involves no explicit variables.

```
In[4]:= exprod[n_Integer] :=
      Expand[ Last[ Nest[ prod, {x, 1}, n ] ] ]
```

Here a result from running the program.

```
In[5]:= exprod[3]
Out[5]= 2 x + 3 x2 + x3
```